

Running Achilles on Your CDM

2025-12-02

Contents

1	Introduction	1
2	General Approach	1
2.1	SQL Only Mode	2
2.2	Logging	2
2.3	Verbose Mode	2
2.4	Preparation for running Achilles	2
3	Achilles Parameters (Both Modes)	2
3.1	Staging Table Prefix	2
3.2	Source Name	3
3.3	Create Table	3
3.4	Limiting the Analyses	3
3.5	Cost Analyses	3
3.6	Small Cell Count	3
3.7	Drop Scratch Tables	3
3.8	Create Indices	3
3.9	Return Value	3
4	Running Achilles: Single-Threaded Mode	3
5	Running Achilles: Multi-Threaded Mode	4
6	Post-Processing	4
6.1	Creating Indices	4
6.2	Dropping All Staging Tables (Multi-threaded only)	5
7	Acknowledgments	5

1 Introduction

In this vignette we cover how to run the Achilles package on your Common Data Model (CDM) database in order to characterize the dataset. The characterizations can help you learn more about your dataset's features and limitations.

It is a best practice for all OHDSI sites to run Achilles on their CDM datasets to ensure researchers can evaluate study feasibility and contextualize study results.

2 General Approach

The Achilles package consists of:

1. The **achilles** function runs a set of SQL scripts to characterize the domains and concepts of the CDM.

2. The **createIndices** function creates table indices for the achilles tables, which can help improve query performance.
3. The **getAnalysisDetails** function provides descriptions about the full set of Achilles analyses.
4. The **dropAllScratchTables** function is useful only for multi-threaded mode. It can clear any leftover staging tables.

2.1 SQL Only Mode

In most Achilles functions, you can specify `sqlOnly = TRUE` in order to produce the SQL without executing it, which can be useful if you'd like to examine the SQL closely or debug something. The SQL files are stored in the `outputFolder`.

2.2 Logging

File and console logging is enabled across most Achilles functions. The status of each step is logged into files in the `outputFolder`. You can review the files in a common text editor.

2.3 Verbose Mode

The `verboseMode` parameter can be set to `FALSE` if you'd like less details about the function execution to appear in the console. Either way, all details are written to the log files. By default, this is set to `TRUE`.

2.4 Preparation for running Achilles

In order to run the package, you will need to determine if you'd like the Achilles tables and staging tables to be stored in schemas that are separate from your CDM's schema (recommended), or within the same schema as the CDM.

2.4.1 Multi-Threaded vs Single-Threaded

As the **achilles** functions can run independently, we have added a multi-threaded mode to allow for more than 1 SQL script to execute at a time. This is particularly useful for massively parallel processing (MPP) platforms such as Amazon Redshift and Microsoft PDW. It may not be beneficial for traditional SQL platforms, so only use the multi-threaded mode if confident it can be useful.

Further, while multiple threads can help performance in MPP platforms, there can be diminishing returns as the cluster has a finite number of concurrency slots to handle the queries. A rule of thumb: most likely you should not use more than 10.

In the multi-threaded mode, all scripts produce permanent staging tables, whereas in the single-threaded mode, the scripts produce temporary staging tables. In both, the staging tables are merged to produce the final Achilles tables.

3 Achilles Parameters (Both Modes)

The following sub-sections describe the optional parameters in **achilles** that can be configured, regardless of whether you run the function in single- or multi-threaded mode.

3.1 Staging Table Prefix

To keep the staging tables organized, the **achilles** function will use a table prefix of "tmpach" by default, but you can choose a different one using the `tempAchillesPrefix` parameter. This is useful for database platforms like Oracle, which limit the length of table names.

3.2 Source Name

The `sourceName` parameter is used to assign the name of the dataset to the Achilles results. If you set this to `NULL`, the **achilles** function will try to obtain the source name from the `CDM_SOURCE` table.

3.3 Create Table

The `createTable` parameter, when set to `TRUE`, drops any existing Achilles results tables and builds new ones. If set to `FALSE`, these tables will persist, and the **achilles** function will just insert new data to them.

3.4 Limiting the Analyses

By default, the **achilles** function runs all default analyses detailed in the `getAnalysisDetails` function. However, it may be useful to focus on a subset of analyses rather than running the whole set. This can be accomplished by specifying analysis Ids in the `analysisIds` parameter.

3.5 Cost Analyses

By default, the **achilles** function does not run analyses on the `COST` table(s), as they can be very time-consuming, and are not critical to most OHDSI studies. However, you can choose to run these analyses by setting `runCostAnalysis` to `TRUE`. The cost analyses are conditional on the CDM version. If using CDM v5.0, then the older cost tables are queried. If using any version after 5.0, the unified cost table is queried.

3.6 Small Cell Count

To avoid patient identification, you can establish the minimum cell size that should be kept in the Achilles tables. Cells with small counts (less than or equal to the value of the `smallCellCount` parameter) are deleted. By default, this is set to 5. Set to 0 for complete summary without small cell count restrictions.

3.7 Drop Scratch Tables

See the Post-Processing section to read about how to run this step separately

This parameter is only necessary if running in multi-threaded mode

The `dropScratchTables` parameter, if set to `TRUE`, will drop all staging tables created during the execution of **achilles** in multi-threaded mode.

3.8 Create Indices

See the Post-Processing section to read about how to run this step separately

The `createIndices` parameter, if set to `TRUE`, will result in indices on the Achilles results tables to be created in order to improve query performance.

3.9 Return Value

When running **achilles**, the return value, if you assign a variable to the function call, is a list object in which metadata about the execution and all of the SQL scripts executed are attributes. You can also run the function call without assigning a variable to it, so that no values are printed or returned.

4 Running Achilles: Single-Threaded Mode

In single-threaded mode, there is no need to set a `scratchDatabaseSchema`, as temporary tables will be used.

```

connectionDetails <- createConnectionDetails(
  dbms = "postgresql",
  server = "localhost/synpuf",
  user = "cdm_user",
  password = "cdm_password"
)

achilles(
  connectionDetails = connectionDetails,
  cdmDatabaseSchema = "cdm",
  resultsDatabaseSchema = "results",
  outputFolder = "output"
)

```

5 Running Achilles: Multi-Threaded Mode

In multi-threaded mode, you need to specify `scratchDatabaseSchema` and use `> 1` for `numThreads`.

```

connectionDetails <- createConnectionDetails(
  dbms = "postgresql",
  server = "localhost/synpuf",
  user = "cdm_user",
  password = "cdm_password"
)

achilles(
  connectionDetails = connectionDetails,
  cdmDatabaseSchema = "cdm",
  resultsDatabaseSchema = "results",
  scratchDatabaseSchema = "scratch",
  numThreads = 5,
  outputFolder = "output"
)

```

6 Post-Processing

This section describes the usage of standalone functions for post-processing that can be invoked if you did not use them in the **achilles** function call.

6.1 Creating Indices

Not supported by Amazon Redshift or IBM Netezza; function will skip this step if using those platforms

To improve query performance of the Achilles results tables, run the **createIndices** function.

```

connectionDetails <- createConnectionDetails(
  dbms = "postgresql",
  server = "localhost/synpuf",
  user = "cdm_user",
  password = "cdm_password"
)

createIndices(
  connectionDetails = connectionDetails,

```

```

resultsDatabaseSchema = "results",
outputFolder = "output"
)

```

6.2 Dropping All Staging Tables (Multi-threaded only)

If the **achilles** execution has errors, or if you did not enable this step in the call to these functions, use the `dropAllScratchTables` function.

The `tableTypes` parameter can be used to specify which batch of staging tables to drop (“achilles”).

```

connectionDetails <- createConnectionDetails(
  dbms = "postgresql",
  server = "localhost/synpuf",
  user = "cdm_user",
  password = "cdm_password"
)

dropAllScratchTables(
  connectionDetails = connectionDetails,
  scratchDatabaseSchema = "scratch", numThreads = 5
)

```

7 Acknowledgments

Considerable work has been dedicated to provide the **Achilles** package.

```

citation("Achilles")

```

```

#> To cite package 'Achilles' in publications use:

```

```

#>

```

```

#> DeFalco F, Ryan P, Schuemie M, Huser V, Knoll C, Londhe A, Abdul-Basser T, Molinaro A (2025). _Achilles:

```

```

#> Characterization_. R package version 1.8.0, https://github.com/OHDSI/Achilles, <https://ohdsi.github.io/Achilles/>

```

```

#>

```

```

#> A BibTeX entry for LaTeX users is

```

```

#>

```

```

#> @Manual{,

```

```

#>   title = {Achilles: Achilles Data Source Characterization},

```

```

#>   author = {Frank DeFalco and Patrick Ryan and Martijn Schuemie and Wojtech Huser and Chris Knoll and

```

```

#>   year = {2025},

```

```

#>   note = {R package version 1.8.0, https://github.com/OHDSI/Achilles},

```

```

#>   url = {https://ohdsi.github.io/Achilles/},

```

```

#> }

```